

Exercise 19 — Documentation Navigation for FastAPI

Exercise Description

Use AI to navigate the FastAPI documentation efficiently: build a reading roadmap (Part 1), deep-dive a complex feature (Part 2), translate abstract docs into code (Part 3), and build a blog API mini-application primarily from the docs (Part 4).

Files: - `app.py` — the blog API (users, posts, comments, search). - `test_app.py` — 7 tests, all passing.

Part 1 — Documentation Reading Roadmap

An effective reading order for a newcomer building a REST API quickly: 1. **Tutorial - First Steps / Path Parameters / Query Parameters** — the request basics. 2. **Request Body + Pydantic models** — validation and schemas (the core value). 3. **Response Model + Status Codes** — shaping and typing responses. 4. **Handling Errors** — `HTTPException`. 5. **Bigger Applications - APIRouter** — structuring beyond one file. 6. **Dependencies** — reuse and cross-cutting concerns. 7. **Security** — OAuth2/JWT when you need auth.

The five most important sections for a quick REST API: request body/Pydantic, path & query parameters, response model, error handling, and APIRouter.

Part 2 — Deep Dive: Dependency Injection (`Depends`)

Notes distilled from the docs: a dependency is any callable declared with `Depends(...)`; FastAPI resolves it per request, caches it within the request, and injects the result. Use it for shared validation, DB sessions, pagination params, and auth (`get_current_user`). When *not* to use it: trivial one-off logic that isn't reused. In this app, `_get_post_or_404` is a plain helper (reused imperatively); a larger app would express it as a dependency so it also documents itself in OpenAPI.

Part 3 — Concept → Code Translation

- **APIRouter** (docs concept) → three routers (`users_router`, `posts_router`, `search_router`) mounted via `app.include_router`, giving grouped, tagged docs.
- **Pydantic request/response models** → `UserCreate/User`, `PostCreate/Post`, `CommentCreate/Comment`, with `response_model=` on each route so responses are validated and documented.
- **Status codes** → 201 on create, 204 on delete, 409/400/404 on the error paths.
- **Query validation** → `Query(..., min_length=1)` on the search term.

Part 4 — Blog API (built incrementally)

Feature	Endpoints
Users	POST <code>/users</code> (unique username, email format validated)
Posts CRUD	POST/GET <code>/posts</code> , GET/PUT/DELETE <code>/posts/{id}</code> (author must be a known user)
Comments	POST <code>/posts/{id}/comments</code> , GET <code>/posts/{id}/comments</code>
Search	GET <code>/search?q=</code> (case-insensitive match on title or body)

Test run:

7 passed in 1.77s

Covers registration + conflict, email validation (422), author validation (400), full post CRUD with 404, comments (incl. comment-on-missing-post 404), and search by title and body.

(Note: a full implementation would add authentication — see Exercise 18 — so that "author" comes from the logged-in user rather than the request body.)

Submission for Exercise 19 — Documentation Navigation for FastAPI

1. Personalised documentation reading guide: the 7-step order and the five must-read sections for a quick REST API (Part 1).

2. Deep-dive notes on a complex feature: dependency injection (`Depends`) — what it is, when to use it, when not to, and how it self-documents (Part 2).

3. Concept-to-code reference: `APIRouter` → routers, `Pydantic` → request/response models, status codes, and query validation, each mapped to concrete lines in `app.py`.

4. Blog mini-application: users, posts (CRUD), comments, and search, built from the docs' recommended patterns and verified by `test_app.py` — **7 tests passing** — with a note on how the docs informed the `APIRouter` structure and where auth would slot in.